MODULE V

GREEDY STRATERGY.

A greedy algorithm always make the choice that looks best at the moment. That is it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution. Greedy algorithms do not always yield optimal solution, but for many problems they do.

The Control Abstraction.

In the control abstraction array a[1:n] stores the n inputs. The solution set is initialized as NULL. With the help of some selection criteria an input x is chosen from the array. 'Feasible' is a boolean function that checks wheather

adding x to the solution set will still keep the solution feasible. If yes the input x will be added to the solution set.

```
Algorithm Greedy(n)
{ //a[1:n] stores the n inputs
  solution = ϕ   // initialization.
  for i=1 to n
  {
     x = select(a)
     if feasible (solution, x)
          solution = union (solution, x)
  }
}
```

The Knapsack Problem.

Given a knapsack of capacity m and n items to be considered to fill the knapsack with. The item i is associated with a weight w_i and profit p_i. If a fraction x_i where $0 \le x_i \le 1$ is chosen, a profit x_i can be ~~chosen~~ earned. The maximum capacity constraint of the knapsack should not be violated. ~~The maximum capacity constraint of the knapsack should not~~. The knapsack problem gives a filling of the knapsack that maximize the profit within the capacity limit, subject to the following conditions,

minimize weight, $\sum_{0 \le x_i \le 1} w_i x_i \le m$

maximize profit. $\sum_{0 \le x_i \le 1} p_i x_i$ should be maximum.

$0 \le x_i \le 1$ and $1 \le i \le n$.

Q. $n = 3$ $m = 20$. $n \rightarrow$ no of items

$p = \{25, 24, 20\}$. $m \rightarrow$ max capacity of knapsack.

$w = \{18, 15, 10\}$.

⇒ Find profit/weight for each element.

object	1	2	3
profit	25	24	20
weight	18	15	10
P/w	1.38	1.6	2.
x	0	10/15	1

20 − 10 = 10

10 − 10 = 0.

weight,

$\sum w_i x_i = 0 \times 18 + \frac{2}{3} \times 15 + 1 \times 10$

$= \underline{\underline{20}}$.

profit,

$\sum p_i x_i = 0 \times 25 + \frac{2}{3} \times 24 + 1 \times 20$

$= \underline{\underline{36}}$.

⇒ If items are selected on basis of profit.

$x\ [1 \quad 2/15 \quad 0]$

$\sum w_i x_i = 1 \times 18 + \frac{2}{15} \times 24 + 0 \times 20$

$= 20$

$\sum p_i x_i = 1 \times 25 + \frac{2}{15} \times 24 + 0$

$= 28.2$.

⇒ If items are selected based on lowest capacity first.

$x\ [0 \quad 10/15 \quad 1]$

$\Sigma w_i x_i = 0 \times 16 + \frac{10}{15} \times 15 + 1 \times 10$
$= 20$

$\Sigma p_i x_i = 0 \times 25 + \frac{10}{15} \times 24 + 1 \times 20$
$= 36$

NOTE

when profit is taken as optimization measure, it gives a sub-optimal solution whereas the other two optimization measures give the optimal solution.

* In knapsack problem, profit / capacity ratio is ~~proposed~~ found to be the best optimization measure.

Q $n = 7$ $m = 15$

$P = \{10, 5, 15, 7, 6, 18, 3\}$

$W = \{2, 3, 5, 7, 1, 4, 1\}$

object	1	2	3	4	5	6	7
profit	10	5	15	7	6	18	3
weight	2	3	5	7	1	4	1
P/w	5	1.6	3	1	6	4.5	3
fraction x	1	2/3	1	0	1	1	1

$\Sigma w_i x_i = 1 \times 2 + \frac{2}{3} \times 3 + 1 \times 5 + 0 \times 7$
$+ 1 \times 1 + 1 \times 4 + 1 \times 1$
$= 2 + 2 + 5 + 1 + 4 + 1$
$= 15$

$15 - 1 = 14$
$14 - 2 = 12$
$12 - 4 = 8$
$8 - 5 = 3$
$3 - 1 = 2$

$\Sigma p_i x_i = 1 \times 10 + \frac{2}{3} \times 5 + 1 \times 15 + 0 \times 7$
$+ 1 \times 6 + 1 \times 18 + 1 \times 3$
$= 10 + 3.33 + 15 + 6 + 18 + 3$
$= \cancel{82.33}$ 55.33

18
15
10
9
52

Q $n = 4$ $m = 20$

$p = \{9, 10, 11, 6\}$

$w = \{11, 10, 7, 14\}$

object	1	2	3	4
profit	9	10	11	6
weight	11	10	7	14
P/w	0.81	1	1.57	0.42
ratio x_i	3/11	1	1	0

$20 - 7 = 13$

$13 - 10 = 3$

$$\sum_{i=1}^{w} p_i x_i = \frac{3}{11} \times 11 + 1 \times 10 + 1 \times 7 + 0 \times 14$$

$$= 3 + 10 + 7$$

$$= 20$$

$$\sum p_i x_i = \frac{3}{11} \times 9 + 1 \times 10 + 1 \times 11 + 0 \times 6$$

$$= 2.45 + 10 + 11$$

$$= 23.45$$

Complexity

Complexity is the order of n, disregarding the time for sorting objects.

Algorithm Greedy knapsack(m, n)

```
{   // m capacity of knapsack
    // n total number of objects
    // P[1:n] stores the profits
    // w[1:n] stores the weights
    // The objects are sorted in decreasing
       order of profit/capacity ratio.

    U = m
    for(i=1 to n)
    {   x[i] = 0
    }
    for(i=1 to n)
    {   if w[i] > U break
        else
            x[i] = 1
            U = U - w[i]
    }
    if i<n , x[i] = U / w[i]
}
```

Minimum Spanning Tree

```
Generic MST (G, E)
{
    MST = ϕ
    while MST is a minimum spanning tree
    {
        select an edge (u, v)
        if feasible (u, v)
            MST = MST (u, v)
    }
}
```

Both prims and kruskal are following greedy strategy.

They take one decision at a time from the several possible decisions. That is a minimum spanning tree grows by adding one edge at a time. Both Prims and kruskals always give optimal solution